

Master Informatique

NAVIGATION AUTONOME EN ENVIRONNEMENT INCERTAIN

Rapport
en vue de la validation de l'UE Initiation à la recherche

Année universitaire 2025-2026

Étudiants :
Eyub CELEBIOGLU - Rahma OUANENNAS - Yoann MORAU

Encadrant :
Olivier BUFFET

Décharge de responsabilités

L'Université de Lorraine n'entend donner ni approbation ni improbation aux opinions émises dans ce rapport, ces opinions devant être considérées comme propres à leur auteur.

Remerciements

Nous tenons à remercier l'ensemble des personnes ayant contribué au bon déroulement de ce projet ainsi qu'à la réalisation de ce rapport.

Nous adressons tout d'abord nos remerciements à notre encadrant, Monsieur Olivier Buffet, pour son accompagnement, sa disponibilité et la qualité de ses conseils tout au long de ce travail. Son expertise et ses retours nous ont permis de progresser et d'approfondir notre réflexion.

Nous remercions également l'ensemble des enseignants du Master Informatique pour la qualité de leur formation, ainsi que toutes les personnes qui nous ont soutenus et conseillés durant ce projet.

Table des matières

Décharge de Responsabilités **2**

Remerciements **3**

Introduction **6**

I Modèle de robot mobile employé **7**

II Algorithme RRT (Rapidly-exploring Random Tree) **7**
A Intérêt du RRT . . . 7
B Fonctionnement du RRT . . . 7
C Variante du RRT : RRT* . . . 8
D Fonctionnement RRT* . . . 9
E Mise en œuvre . . . 9
1 Mise en œuvre du RRT . . . 9
2 Mise en œuvre du RRT* . . . 10
3 Mise en œuvre du RRTSC . . . 10
F Expérimentation . . . 12
G Présentation des cartes de test . . . 12
H Résultats et Analyse . . . 12
1 Présentation des résultats . . . 12
2 Analyse . . . 16
I Conclusion . . . 17

III Apprentissage par renforcement : MDP et Q-Learning **17**
A Apprentissage par renforcement . . . 17
B Processus de Décision Markoviens (MDP) . . . 18
C Définition du Q-learning : . . . 18
D Mise en œuvre du Q-learning . . . 19
1 Architecture du système . . . 19
2 Processus d'apprentissage . . . 20
3 Fonction de récompense . . . 21
4 Expérimentations . . . 21
E Analyse et discussion . . . 23
1 Impact de l'exploration et de l'exploitation . . . 23
2 Convergence de la fonction de valeur . . . 23
3 Effet de la fonction de récompense . . . 23
4 Amélioration des trajectoires . . . 24
5 Limites du modèle . . . 24

IV Apprentissage par renforcement profond : Deep Q-Learning (DQN) **24**
A Introduction au DQN . . . 24
B Contribution (Notre approche et évolution du modèle) . . . 25
1 L'évolution de la perception du robot . . . 25
2 L'architecture du réseau de neurones . . . 26
3 Le gonflement des obstacles . . . 27

4 Le système de récompenses 27
5 La mécanique de traumatisme 28
C Expérimentation (Analyse et résultats) 28
1 Protocole d'entraînement 28
2 Progression de l'entraînement 28
3 Résultats par carte 30
4 Évolution de l'erreur du réseau 31
5 Visualisation du comportement 31
D Pistes d'amélioration 32

Conclusion **34**

Bibliographie **35**

Résumé **40**

INTRODUCTION

Dans le domaine de la **robotique mobile**, la **navigation autonome** est un défi technique et technologique majeur. Pour un robot mobile, la navigation consiste à se **localiser** et à **planifier un chemin** dans un environnement potentiellement encombré d'obstacles [5]. Elle se distingue du **contrôle de trajectoire** (niveau plus bas), qui consiste à exécuter ce chemin en tenant compte de la dynamique réelle du robot et d'éventuelles incertitudes physiques (sol glissant, capteurs imprécis, obstacles imprévus).

Ces deux aspects sont classiquement traités séparément : un planificateur calcule une trajectoire en supposant un modèle simplifié du robot, puis un contrôleur l'exécute en s'adaptant aux écarts entre le modèle et la réalité. Dans un **environnement incertain**, ce contrôleur doit être robuste et capable de réagir à l'imprévu.

Une piste prometteuse pour concevoir hors-ligne un contrôleur à la fois rapide à exécuter et robuste aux incertitudes est l'**apprentissage par renforcement**. Il permet à l'agent de construire une **politique d'action** par essais et erreurs, sans modèle explicite de l'environnement. Cependant, apprendre directement à naviguer sans guidage préalable serait coûteux : l'espace de recherche serait trop grand pour une exploration efficace. C'est pourquoi nous proposons d'exploiter des trajectoires issues d'un algorithme de planification pour guider cet apprentissage.

> **Comment exploiter une trajectoire générée par un algorithme de planification déterministe afin de guider un algorithme d'apprentissage par renforcement et améliorer l'apprentissage de la navigation d'un véhicule dans un environnement incertain ?**

Nous proposons deux approches complémentaires pour utiliser l'apprentissage par renforcement. La première vise à **apprendre un contrôleur qui optimise le suivi d'une trajectoire précise** fournie en entrée par un algorithme de planification. La seconde vise à **apprendre un contrôleur générique**, indépendant de la trajectoire, capable de suivre n'importe quel chemin planifié.

Ce rapport est organisé comme suit. La section I présente le modèle du robot mobile et de son environnement. La section II introduit l'algorithme de planification RRT ainsi que deux variantes que nous proposons. La section III introduit le formalisme des processus de décision markoviens et l'algorithme du Q-learning, ainsi que l'approche que nous proposons pour optimiser le contrôle d'une trajectoire donnée. La section IV décrit l'algorithme DQN, qui combine le Q-learning avec des réseaux de neurones profonds, et l'approche que nous proposons pour apprendre un contrôleur générique de suivi de trajectoire.

I Modèle de robot mobile employé

Nous supposons un environnement 2D rectangulaire dans lequel un robot circulaire doit naviguer d'un point de départ vers un objectif tout en évitant des obstacles rectangulaires fixes. Ces obstacles pourraient être de formes plus génériques, mais nous nous limitons ici aux rectangles pour simplifier la détection de collision. Un exemple d'environnement est illustré en figure 2.

Le robot est modélisé avec une cinématique simple. Il possède une **vitesse** v et un **angle d'orientation** θ. À chaque pas de simulation, sa position (x, y) est mise à jour selon les équations suivantes :

$$x \leftarrow x + \cos(\theta) \cdot v \qquad y \leftarrow y + \sin(\theta) \cdot v$$

Le robot dispose de quatre actions motrices : **accélérer** (augmenter v d'un incrément fixe), **freiner** (réduire v), **tourner à gauche** (augmenter θ d'un incrément fixe) et **tourner à droite** (diminuer θ). Le simulateur ainsi conçu a d'abord été testé avec une commande manuelle au clavier avant d'être interfacé avec les algorithmes de planification et d'apprentissage.

II Algorithme RRT (Rapidly-exploring Random Tree)

A Intérêt du RRT

Cet algorithme a été proposé par **Steven M. LaValle** [1] et **James J. Kuffner Jr.** [2]. L'algorithme **RRT** a pour but d'explorer des **espaces de configurations** de grande dimension. Un espace de configurations représente l'ensemble des états possibles d'un robot : pour un bras robotique à plusieurs articulations, cet espace peut être de très haute dimension ; pour un robot mobile avec remorque, il inclut la position et l'orientation de chaque segment.

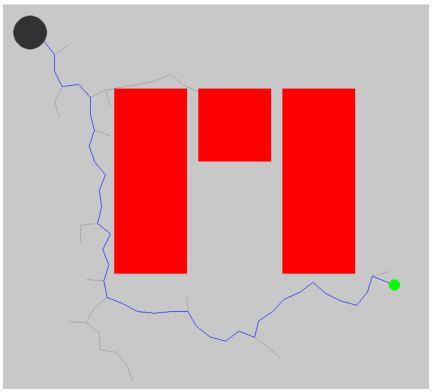
Dans notre cas, nous travaillons dans un espace de configurations à trois dimensions (x, y, θ), représentant la position et l'orientation du robot dans le plan. Nous n'intégrons pas la vitesse dans cet espace, celle-ci étant contrôlée séparément par apprentissage par renforcement. Ce choix permet de **réduire l'espace de recherche** et donc d'accélérer la planification de chemin, tout en conservant les contraintes cinématiques essentielles du véhicule. Il est important de noter que l'algorithme RRT décrit dans la suite est **générique** et s'applique à tout espace de configurations, quelle que soit sa dimension.

On retrouve cet algorithme notamment dans le domaine de la **robotique autonome** et de la **planification de chemin**, par son efficacité à générer des trajectoires dans des environnements encombrés d'obstacles.

B Fonctionnement du RRT

À partir d'un point initial, on génère aléatoirement un **point** dans l'espace. On recherche ensuite le **nœud le plus proche** de ce point dans l'**arbre**, puis on étend l'arbre dans sa direction avec un **pas de taille fixe**. Si cette extension ne provoque pas de **collision**, on ajoute ce nouveau point à l'arbre en le reliant à son parent.

On répète ce processus de manière itérative, ce qui permet de construire progressivement un arbre qui explore l'espace. L'algorithme s'arrête lorsque la destination est atteinte ou lorsqu'un certain **critère d'arrêt** est vérifié. Le **chemin final** correspond alors à une suite de branches reliant le point de départ au point d'arrivée dans l'arbre.



(a) Chemin trouvé par RRT

FIGURE 1 – Étapes de construction d'un RRT. En gris les branches explorées, en bleu le chemin final, en rouge les obstacles.

"Le pseudo-code suivant est inspiré de [1]"

Algorithm 1 Construction d'un RRT

Entrée : Configuration initiale q_{init}, nombre d'itérations K, pas Δq
Sortie : Graphe G
$G \leftarrow \text{init}(q_{init})$
for $k = 1$ à K **do**
    $q_{rand} \leftarrow \text{RAND_CONF}()$
    $q_{near} \leftarrow \text{NEAREST_VERTEX}(q_{rand}, G)$
    $q_{new} \leftarrow \text{NEW_CONF}(q_{near}, q_{rand}, \Delta q)$
    G.add_vertex(q_{new})
    G.add_edge(q_{near}, q_{new})
end for
return G

Description des fonctions `RAND_CONF` génère une configuration aléatoire q_{rand} dans l'espace des configurations C. Cette fonction peut être remplacée par `RAND_FREE_CONF`, qui ne génère que des configurations appartenant à l'espace libre C_{free}, en rejetant celles situées dans les obstacles C_{obs}.

`NEAREST_VERTEX` parcourt l'ensemble des nœuds du graphe G et retourne le nœud le plus proche de q_{rand} selon une métrique donnée.

`NEW_CONF` génère une nouvelle configuration q_{new} en avançant d'une distance Δq depuis q_{near} en direction de q_{rand}.

C Variante du RRT : RRT*

Il existe une quarantaine de variations et d'améliorations de l'algorithme **RRT**. Nous nous focalisons ici sur l'une des premières variantes ayant su **optimiser la trajectoire** générée. En effet, le RRT classique ne renvoie jamais une trajectoire optimale en raison de sa construction aléatoire. C'est ici que la variante **RRT*** prend tout son sens, proposée par **Sertac Karaman**

et **Emilio Frazzoli** [3]. Son principal avantage est que, lorsque le nombre de nœuds **tend vers l'infini**, l'algorithme converge vers le **chemin le plus court** possible.

D Fonctionnement RRT*

L'algorithme repose sur le même principe que le RRT, mais introduit deux mécanismes supplémentaires.

Le premier est la sélection du **meilleur parent**. Lors de l'ajout d'un nouveau nœud, l'algorithme examine l'ensemble des voisins situés dans un rayon fixe et sélectionne celui dont le **coût de chemin est le plus faible**, plutôt que le simple nœud le plus proche.

Le second est le **recâblage de l'arbre**. Après avoir connecté le nouveau nœud à son meilleur parent, l'algorithme vérifie si les voisins pourraient bénéficier d'un **coût réduit** en passant par ce nouveau nœud. Si c'est le cas, leur connexion est mise à jour, produisant des trajectoires plus lisses et plus courtes. En contrepartie, ces deux opérations augmentent le temps de calcul.

Algorithm 2 Construction d'un RRT*

Entrée : Configuration initiale q_{init}, nombre d'itérations K, pas Δq, rayon de voisinage r
Sortie : Graphe G
$G \leftarrow \text{init}(q_{init})$
for $k = 1$ à K **do**
    $q_{rand} \leftarrow \text{RAND_CONF}()$
    $q_{near} \leftarrow \text{NEAREST_VERTEX}(q_{rand}, G)$
    $q_{new} \leftarrow \text{NEW_CONF}(q_{near}, q_{rand}, \Delta q)$
    $Q_{voisins} \leftarrow \text{NEIGHBORS}(q_{new}, G, r)$
    $q_{best} \leftarrow \text{BEST_PARENT}(q_{new}, Q_{voisins})$
    $G.\text{add_vertex}(q_{new})$
    $G.\text{add_edge}(q_{best}, q_{new})$
    $G \leftarrow \text{REWIRE}(G, q_{new}, Q_{voisins})$
end for
return G

`BEST_PARENT` examine les nœuds q' du voisinage $Q_{voisins}$ et retourne le nœud q_{best} minimisant $\text{cost}(q') + d(q', q_{new})$.

`REWIRE` parcourt les voisins et reconnecte ceux pour lesquels $\text{cost}(q_{new}) + d(q_{new}, q') < \text{cost}(q')$, en remplaçant leur parent par q_{new}.

E Mise en œuvre

1 Mise en œuvre du RRT

Notre mise en œuvre prend en entrée les paramètres suivants : la **taille de la carte**, l'**angle maximal** qu'une branche peut prendre pour éviter des changements de direction trop brusques qu'un véhicule ne pourrait pas effectuer à vitesse importante, la **longueur des branches**, la **liste des obstacles**, le **nombre d'itérations**, le **point de départ** et le **point d'arrivée**. L'espace de configurations exploré est (x, y, θ) comme décrit en section II.A.

Détection de collision Une méthode classique pour un véhicule de forme circulaire comme le nôtre consiste à **élargir les obstacles** d'une valeur égale au rayon du robot. Ainsi, il suffit de tester si le **centre du robot** touche ces obstacles élargis, ce qui simplifie le test de collision.

Pour vérifier qu'un déplacement entre deux points ne rencontre pas d'obstacle, on **discrétise le segment** parcouru par le centre du robot pixel par pixel et on teste chaque point intermédiaire.

Recherche du nœud le plus proche La recherche du nœud le plus proche parcourt actuellement **tous les nœuds** de l'arbre, ce qui devient de plus en plus lent à mesure que l'arbre grandit. Il serait possible d'accélérer cette opération en stockant les nœuds dans une structure de données plus efficace, comme un **KD-tree** ou un **cover tree** [4], qui permettent une recherche en $O(\log n)$ au lieu de $O(n)$.

2 Mise en œuvre du RRT*

La mise en œuvre du RRT* repose sur la même base que le RRT, avec deux ajouts principaux. La sélection du meilleur parent et le recâblage intègrent tous deux une **vérification angulaire** : la différence entre l'angle de la nouvelle branche et celui du nœud parent est normalisée dans $[-\pi, \pi]$ pour éviter les discontinuités de $\arctan 2$, puis comparée à l'angle maximal autorisé θ. Cela garantit que le recâblage ne crée pas de connexions avec des virages impossibles à suivre pour le robot.

Après chaque recâblage, le coût de **tous les descendants** du nœud reconnecté est mis à jour de manière récursive pour propager le nouveau coût dans l'arbre.

3 Mise en œuvre du RRTSC

Le **RRTSC (Short-Cut)** est une **contribution propre à ce projet**, ne s'appuyant sur aucune implémentation existante. L'idée est née du constat que le RRT* nécessite un grand nombre d'itérations pour converger, alors qu'une phase de post-traitement sur le chemin brut du RRT classique peut produire des résultats comparables bien plus rapidement.

Le RRTSC reprend la structure du RRT classique et y ajoute une **phase de raccourcis** décrite par l'algorithme suivant :

Algorithm 3 Construction d'un RRT avec Raccourcis (RRTSC)

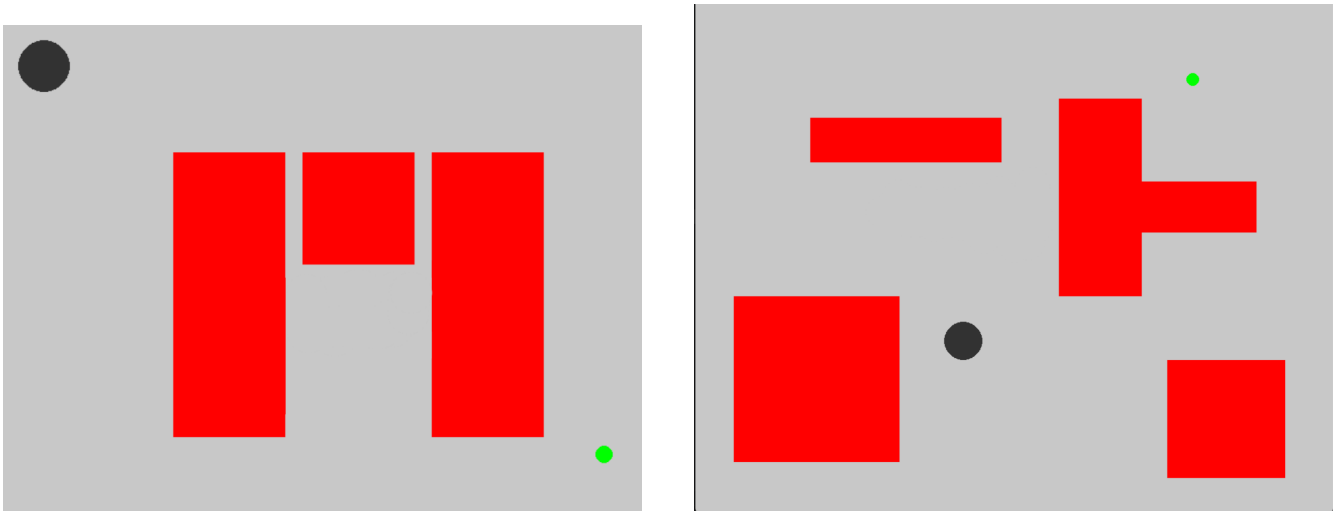
Entrée : q_{init}, K, Δq, θ, rayon disque $r < \Delta q$, seuil ϵ
Sortie : Chemin P
$G \leftarrow \text{init}(q_{init})$, construire RRT classique
$P \leftarrow \text{RETRACE_PATH}(G, q_{init}, q_{goal})$
$c_{prec} \leftarrow \text{COST}(P)$
repeat
    for chaque nœud $q_i \in P$ **do**
        for chaque multiplicateur $m = 1, 2, 3, \ldots$ **do**
            for chaque angle α autorisé par θ **do**
                $q_{branch} \leftarrow q_i + m \cdot \Delta q \cdot (\cos\alpha, \sin\alpha)$
                if OBSTACLE(q_i, q_{branch}) **then**
                    break
                end if
                for chaque nœud $q_j \in P,\ j > i + 1$ **do**
                    if $\|q_{branch} - q_j\| \leq r$ **then**
                        Mémoriser q_j comme meilleur raccourci
                    end if
                end for
            end for
        end for
        if raccourci trouvé vers q_{best} et angles valides **then**
            $P \leftarrow P[0 : i] + q_{branch} + P[q_{best} :]$
        end if
    end for
    $c_{act} \leftarrow \text{COST}(P), \quad ratio \leftarrow \frac{c_{prec} - c_{act}}{c_{prec}}, \quad c_{prec} \leftarrow c_{act}$
until $ratio < \epsilon$
return P

La vérification angulaire est appliquée dans les deux sens du raccourci : à l'entrée (depuis q_i vers q_{branch}) et à la sortie (depuis q_{branch} vers q_j), garantissant que le raccourci respecte les contraintes cinématiques du robot.

F Expérimentation

Maintenant que les algorithmes sont mis en œuvre, il nous faut les évaluer. Les tests ont été effectués sur deux cartes distinctes (figure 2), avec deux configurations de paramètres chacune, et répétés 100 fois par point de mesure afin d'obtenir des moyennes représentatives.

G Présentation des cartes de test



(a) Map 1 — trois obstacles rectangulaires

(b) Map 2 — cinq obstacles de formes variées

FIGURE 2 – Cartes utilisées pour les tests. En rouge les obstacles élargis du rayon du robot, en vert l'objectif, en noir le robot au point de départ.

La Map 1 présente trois obstacles rectangulaires disposés en couloir, forçant les algorithmes à trouver un passage. La Map 2 est plus complexe avec cinq obstacles de formes et tailles variées, dont une forme en croix réduisant significativement l'espace libre. Les paramètres testés sont :

— $(l = 20,\ \theta = 30)$ et $(l = 30,\ \theta = 25)$ sur chaque carte

— Nombre d'itérations : de 500 à 6000 par pas de 500

Les métriques collectées sont le **temps d'exécution**, le **taux de réussite**, le **nombre de nœuds** dans le chemin final et le **coût** calculé par :

$$C(P) = \sum_{k=0}^{n-1} \sqrt{(x_{k+1} - x_k)^2 + (y_{k+1} - y_k)^2} \tag{1}$$

où P est le chemin, n le nombre de nœuds, et (x_k, y_k) les coordonnées du k-ième nœud.

H Résultats et Analyse

1 Présentation des résultats

Les tableaux 3 et 4 présentent, pour chaque algorithme et chaque configuration, les valeurs moyennes du temps d'exécution (en secondes), du coût du chemin (en pixels), du nombre de nœuds et du taux de réussite (en %). Les graphes 5, 6 et 7 illustrent l'évolution de ces métriques en fonction du nombre d'itérations, avec une courbe par algorithme.

Longeur : 20 Theta 30												
Itérations	Temps RRT	Temps RRTSC	Temps RRT*	Cout RRT	Cout RRTSC	Cout RRT*	Noeuds RRT	Noeuds RRTSC	Noeuds RRT*	Succès RRT* (%	Succès RRTSC	Succès RRT (%)
500	0,0516	0,4466	0,0583	404,9	397,6	455,5	20,5	5,4	15,3	43	38	37
1000	0,0984	1,0406	0,2146	922,8	795,5	903,6	46,6	11,5	30,2	84	76	83
1500	0,1515	1,2459	0,4444	1031,6	973,6	993,7	52,1	13,2	33,6	93	93	92
2000	0,1865	1,3849	0,7811	1059,7	1038,8	1039,7	53,5	14,2	34,9	98	98	95
2500	0,1605	1,3182	1,2016	1089,5	1031,1	1028,8	55	14,8	34,7	98	98	98
3000	0,1477	1,3199	1,7194	1099,1	1034,5	1052,8	55,5	14,8	35,8	100	98	99
3500	0,1587	1,3065	2,3218	1117,5	1047,3	1052,4	56,4	14,8	35,6	100	99	100
4000	0,1861	1,3567	3,022	1105,6	1048,4	1041,3	55,8	13,7	35,6	100	100	100
4500	0,1833	1,3933	3,7977	1120,6	1051,8	1040,1	56,6	14,7	35,8	100	100	100
5000	0,1856	1,2544	4,7037	1120,4	1049,8	1037,3	56,6	14,1	35,4	100	100	100
5500	0,195	1,3368	5,6431	1107,6	1059,5	1037,7	56	15,1	35,6	100	100	100
6000	0,1526	1,3373	6,8191	1106,5	1050,3	1038,7	55,9	14,2	36,2	100	100	100

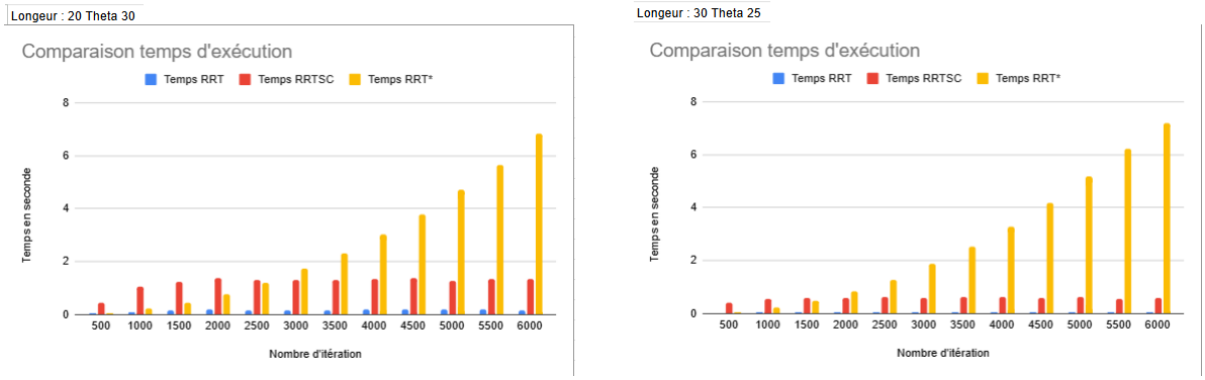
Longeur : 30 Theta 25												
Itérations	Temps RRT	Temps RRTSC	Temps RRT*	Cout RRT	Cout RRTSC	Cout RRT*	Noeuds RRT	Noeuds RRTSC	Noeuds RRT*	Succès RRT (%)	Succès RRTSC	Succès RRT* (%)
500	0,0337	0,4226	0,0618	847,8	811	1003,9	28,7	10,3	34,4	78	75	94
1000	0,0482	0,5714	0,2187	1066,6	1024,8	1045,2	36,2	12,7	35,8	97	95	98
1500	0,058	0,5845	0,484	1077,8	1076,3	1053,4	36,5	13,7	36	98	100	100
2000	0,0505	0,6037	0,8388	1092,8	1075,9	1052,7	37	14	36,1	100	100	100
2500	0,0579	0,6243	1,2877	1100,6	1084,3	1038,7	37,3	14,1	35,6	100	100	100
3000	0,0628	0,601	1,8634	1099,8	1078,6	1038,6	37,2	13,6	35,6	100	100	100
3500	0,0497	0,6131	2,5175	1090,3	1085,5	1039,5	37	13,5	35,8	100	100	100
4000	0,0544	0,6151	3,2879	1100,6	1086,2	1029,8	37,2	13,9	35,5	100	100	100
4500	0,0482	0,5961	4,1666	1101,2	1071,8	1029,8	37,2	13,8	35,5	100	100	100
5000	0,051	0,6353	5,1948	1086,5	1080	1027,8	36,8	13,3	35,4	100	100	100
5500	0,0577	0,5642	6,2319	1097	1077,5	1023,2	37,1	14,1	35,3	100	100	100
6000	0,0483	0,6024	7,1969	1091,6	1078,5	1025,2	36,9	13,6	35,4	100	100	100

FIGURE 3 – Résultats sur la Map 1. Colonnes : itérations, puis pour chaque algorithme (RRT, RRTSC, RRT*) : temps moyen (s), coût moyen (px), nombre de nœuds moyen, taux de réussite (%).

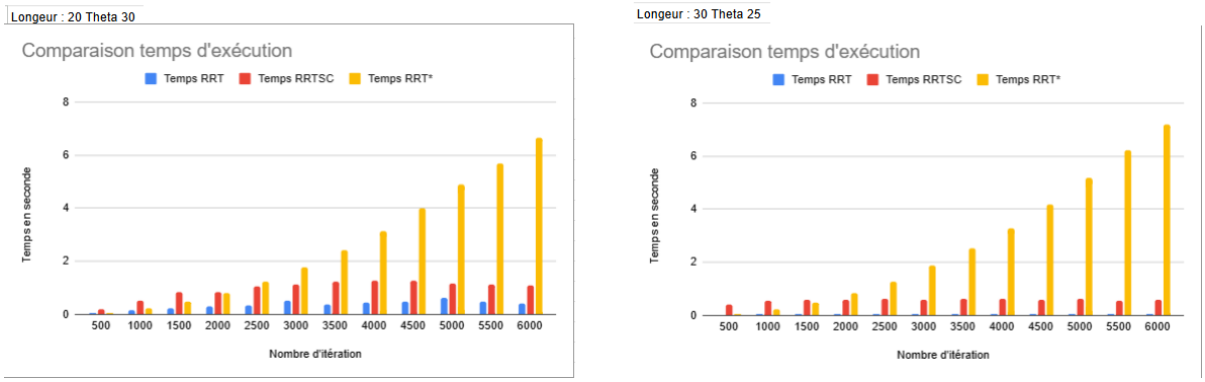
Longeur : 20 Theta 30												
Itérations	Temps RRT	Temps RRTSC	Temps RRT*	Cout RRT	Cout RRTSC	Cout RRT*	Noeuds RRT	Noeuds RRTSC	Noeuds RRT*	Succès RRT (%)	Succès RRTSC	Succès RRT* (%)
500	0,0587	0,1858	0,0668	176,3	171,6	171,7	8,9	2,8	6	22	23	23
1000	0,1693	0,5155	0,216	444	441,4	437,7	22,5	7,7	15,2	53	56	54
1500	0,2439	0,8435	0,4679	676,6	652,7	649,2	34,3	11,5	22,3	80	82	81
2000	0,2902	0,8441	0,8022	752,4	700,2	725	38,1	11,9	25	89	90	89
2500	0,3343	1,0683	1,2485	830,4	732,2	707	42,1	12,4	24,6	98	92	92
3000	0,512	1,1392	1,7849	777,7	774,3	729,3	39,4	12,8	25,5	91	96	92
3500	0,3797	1,228	2,4023	828	788	766,9	42	13,3	26,6	97	98	98
4000	0,4324	1,2668	3,1422	851,3	789,4	764,4	43,1	12,4	26,7	99	97	98
4500	0,4653	1,2686	3,9801	868,1	786,3	768	43,9	12,8	26,9	99	99	99
5000	0,6313	1,1759	4,8911	861,1	802,3	774,5	43,7	12,9	27,1	99	100	100
5500	0,4653	1,1124	5,6772	871,5	811,1	752,5	44,2	13,1	26,6	100	100	100
6000	0,4065	1,0745	6,6355	856,8	787,2	775,8	43,4	13,4	27,5	100	100	100

Longeur : 30 Theta 25												
Itérations	Temps RRT	Temps RRTSC	Temps RRT*	Cout RRT	Cout RRTSC	Cout RRT*	Noeuds RRT	Noeuds RRTSC	Noeuds RRT*	Succès RRT (%)	Succès RRTSC	Succès RRT* (%)
500	0,052	0,2264	0,0598	405,1	439,2	509,8	13,8	6,4	17,6	48	54	62
1000	0,1017	0,3327	0,209	773,9	681,9	644,1	26,3	10,2	22,2	89	84	77
1500	0,1475	0,4331	0,4476	805,4	811,1	811,9	27,4	11,7	28	91	95	99
2000	0,1329	0,5575	0,7704	853	869	783,1	29	12,7	27	99	100	98
2500	0,1149	0,4429	1,1774	854,3	852,8	767,4	29	12,3	26,5	100	100	98
3000	0,1664	0,4756	1,693	865	838,8	782,2	29,4	12,7	27	98	99	100
3500	0,1482	0,4349	2,3255	859,3	864,7	762,6	29,2	13,1	26,4	100	100	100
4000	0,131	0,4914	3,2036	885,7	850,5	774,5	30,1	12,9	26,7	100	100	100
4500	0,1361	0,5385	4,1279	869,9	849,5	766,8	29,6	12,3	26,5	100	100	100
5000	0,1347	0,4776	4,9822	858	854,5	757,4	29,1	12,5	26,3	100	100	100
5500	0,1304	0,5061	5,8754	843,2	842,2	757,4	28,7	12,4	26,2	100	100	100
6000	0,0959	0,486	6,8903	852,1	836,2	745,8	29	13	25,9	100	100	100

FIGURE 4 – Résultats sur la Map 2. Mêmes colonnes que le tableau précédent.

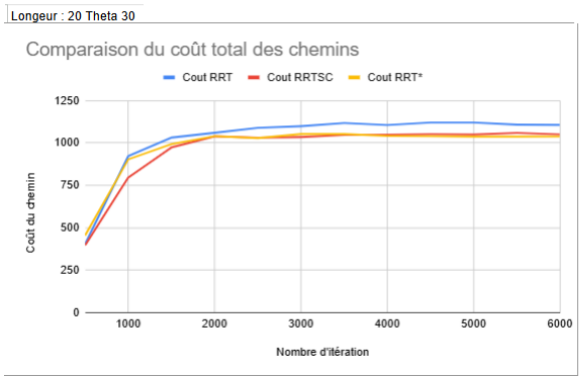


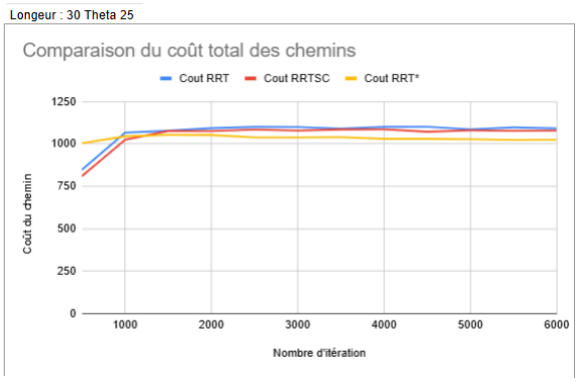
(a) Map 1 — axe X : nombre d'itérations, axe Y : temps moyen d'exécution (s)



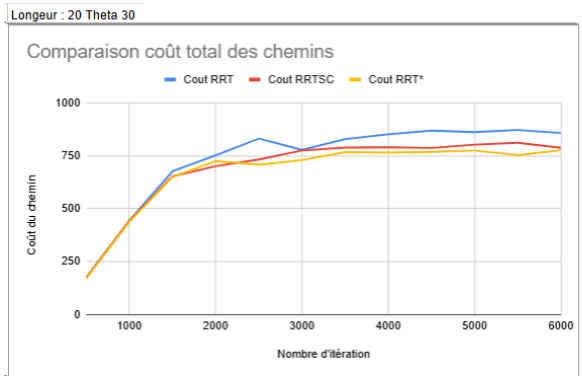
(b) Map 2 — mêmes axes

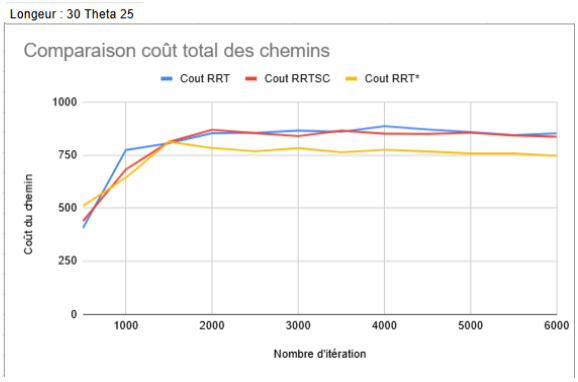
FIGURE 5 – Temps d'exécution moyen. Bleu : RRT, vert : RRTSC, rouge : RRT*.





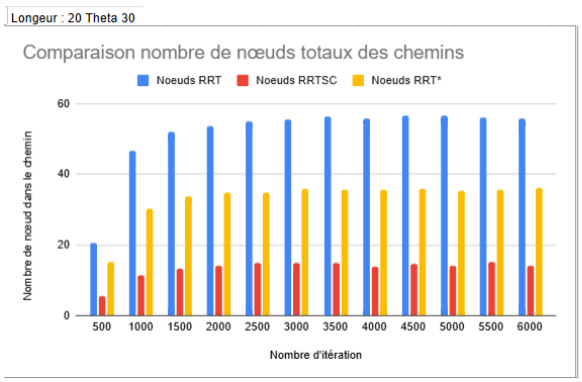
(a) Map 1 — axe X : nombre d'itérations, axe Y : coût moyen du chemin (px)

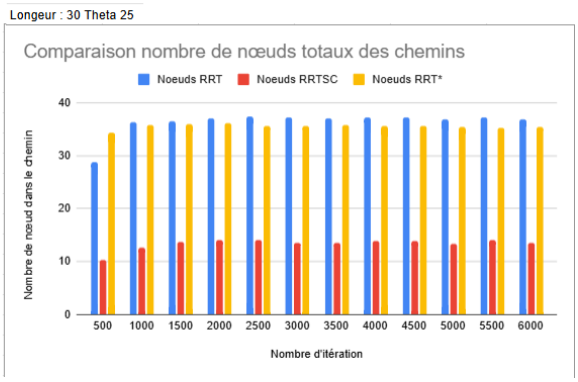




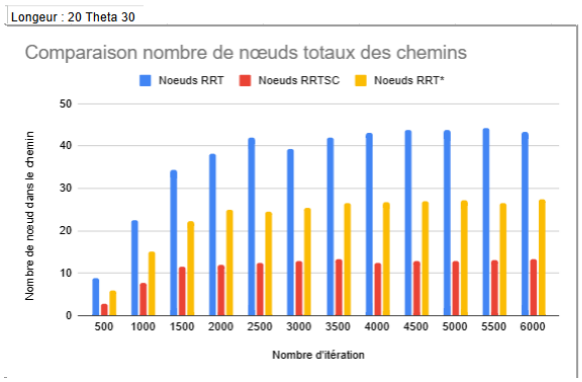
(b) Map 2 — mêmes axes

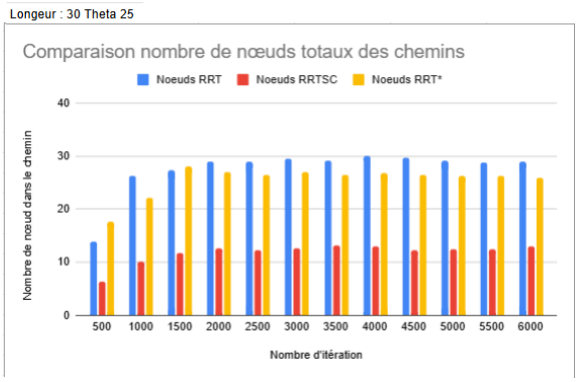
FIGURE 6 – Coût moyen des chemins. Bleu : RRT, vert : RRTSC, rouge : RRT*.





(a) Map 1 — axe X : nombre d'itérations, axe Y : nombre moyen de nœuds





(b) Map 2 — mêmes axes

FIGURE 7 – Nombre moyen de nœuds dans le chemin. Bleu : RRT, vert : RRTSC, rouge : RRT*.

2 Analyse

Temps d'exécution Le **RRT classique** est systématiquement le plus rapide, avec des temps inférieurs à 0,65 s. Le **RRTSC** est plus lent en raison de sa phase de post-traitement, mais son temps reste relativement stable au-delà de 2000 itérations, ce qui indique que l'algorithme de raccourcis converge rapidement. Le **RRT*** présente une croissance quasi-linéaire du temps de calcul, atteignant jusqu'à 7,2 s à 6000 itérations, en raison des opérations de *chooseParent* et *rewiring* effectuées à chaque itération.

Coût des chemins Le **RRT*** produit systématiquement les chemins les moins coûteux à partir d'un nombre d'itérations suffisant, confirmant son optimalité asymptotique. Le **RRTSC** améliore significativement le coût du RRT classique, notamment à faible nombre d'itérations. Le **RRT classique** produit les chemins les plus coûteux, avec un écart qui se réduit à mesure que les itérations augmentent.

Nombre de nœuds Le **RRTSC** se distingue nettement : il réduit le nombre de nœuds de plus de 70% par rapport au RRT classique (14 nœuds contre 55 sur Map 1 à 3000 itérations). Cette réduction est directement liée à la suppression des nœuds intermédiaires, produisant un chemin plus direct et plus exploitable pour le robot. Le RRT* se situe entre les deux avec environ 35 nœuds.

Influence des paramètres Pour le **RRTSC**, une longueur courte ($l = 20$) et un angle large ($\theta = 30$) produisent des chemins de meilleure qualité mais au prix d'un temps d'exécution plus élevé, car un chemin initial plus dense augmente le nombre de combinaisons à tester lors de la phase de raccourcis. Pour le **RRT***, une longueur plus grande ($l = 30$) et un angle plus restreint ($\theta = 25$) permettent au recâblage de connecter des nœuds plus distants en moins d'étapes, favorisant une meilleure optimisation.

Taux de réussite Les trois algorithmes atteignent 100% de réussite à partir de 2000–3000 itérations sur Map 1. Sur Map 2, plus difficile, ce seuil est atteint plus tardivement. Les discontinuités de coût observées à faible nombre d'itérations s'expliquent par un biais statistique : à faible taux de réussite, seuls les cas les plus simples aboutissent, tirant artificiellement le coût moyen vers le bas.

I Conclusion

Ces résultats mettent en évidence un compromis qualité/temps de calcul entre les trois algorithmes. Le **RRT*** est le meilleur choix lorsque l'on dispose du temps nécessaire, produisant les chemins les plus courts. Le **RRTSC** représente le meilleur compromis dans notre contexte : il conserve la rapidité du RRT tout en réduisant drastiquement le nombre de nœuds, produisant des waypoints plus clairs pour guider l'apprentissage par renforcement. Le **RRT classique** reste pertinent uniquement lorsque le temps de calcul est la contrainte principale.

III Apprentissage par renforcement : MDP et Q-Learning

A Apprentissage par renforcement

L'apprentissage par renforcement (Reinforcement Learning, RL) est une approche d'apprentissage dans laquelle un agent apprend à agir dans un environnement par interactions successives et par essais-erreurs.

Contrairement aux méthodes supervisées classiques, l'agent ne dispose pas d'exemples corrects indiquant directement quelle action effectuer. Il reçoit uniquement un signal de récompense fourni par l'environnement après chacune de ses actions. L'objectif est alors d'apprendre un comportement permettant de maximiser les récompenses cumulées à long terme.

Dans ce cadre, l'agent observe un état $s \in S$, choisit une action $a \in A$, puis l'environnement retourne :

— un nouvel état s',

— une récompense $r(s, a, s')$.

L'apprentissage par renforcement repose donc sur l'idée que les actions ayant conduit à des récompenses élevées doivent être favorisées, tandis que celles produisant des pénalités doivent être évitées.

Une difficulté majeure du RL est le compromis entre :

— **exploration** : tester de nouvelles actions afin de découvrir de meilleurs comportements,

— **exploitation** : utiliser les connaissances déjà acquises pour maximiser les récompenses.

Par exemple, dans notre problème de navigation robotique, le robot peut :

— exploiter une trajectoire déjà connue comme efficace,

— ou explorer de nouveaux mouvements susceptibles de produire un chemin plus court ou plus fluide.

Le cadre le plus utilisé pour modéliser l'apprentissage par renforcement est celui des processus de décision markoviens (MDP). Dans ce cadre, les actions peuvent avoir des effets probabilistes et l'agent cherche à apprendre une politique optimale maximisant l'espérance des récompenses futures.

Dans les tâches de navigation comme celle étudiée dans ce projet, l'apprentissage est généralement de type épisodique : chaque épisode débute avec une position initiale du robot et se termine lorsque l'objectif est atteint ou lorsqu'une collision se produit.

B Processus de Décision Markoviens (MDP)

Les méthodes d'apprentissage par renforcement reposent généralement sur le cadre mathématique des processus de décision markoviens (Markov Decision Processes, MDP), introduits notamment par **Bellman**.

Un MDP est défini par le tuple :

$$(S, A, T, R)$$

où :

— S représente l'ensemble des états possibles de l'environnement,

— A représente l'ensemble des actions disponibles,

— $T(s, a, s') = P(s'|s, a)$ est la fonction de transition donnant la probabilité de passer de l'état s à l'état s' après l'action a,

— $R(s, a, s')$ est la fonction de récompense associée à cette transition.

Dans ce cadre, un agent interagit avec son environnement en choisissant des actions selon une politique π.

Une politique est une fonction qui associe à chaque état une action à effectuer :

$$\pi : S \to A$$

L'objectif de l'apprentissage par renforcement est de déterminer une politique optimale maximisant l'espérance des récompenses cumulées futures :

$$E\left[\sum_{t=0}^{\infty} \gamma^t R_t \mid \pi, S_0 = s\right]$$

où $\gamma \in [0, 1]$ est le facteur d'actualisation permettant de pondérer l'importance des récompenses futures.

C Définition du Q-learning :

Dans le cadre des MDP, la fonction de valeur action-état associée à une politique π est définie par :

$$Q^{\pi}(s, a) = E\left[\sum_{t=0}^{\infty} \gamma^t R_t \mid \pi, S_0 = s, A_0 = a\right]$$

Cette fonction représente l'espérance des récompenses cumulées obtenues en exécutant l'action a dans l'état s, puis en suivant la politique π.

Le Q-learning cherche à apprendre la fonction optimale Q^* correspondant à la politique optimale.

Une fois cette fonction apprise, l'agent agit de manière optimale en choisissant l'action maximisant la valeur :

$$a^* = \arg\max_a Q^*(s, a)$$

Pour cela, l'algorithme apprend une fonction appelée fonction de valeur action-état, notée Q(s,a), qui estime l'intérêt de réaliser une action donnée dans un état donné. Plus la valeur de Q(s,a) est élevée, plus cette action est considérée comme avantageuse sur le long terme.

L'algorithme calcule une fonction de valeur action-état : $\mathrm{Q} : \mathrm{S} \times \mathrm{A} \to \mathrm{R}$

Avant que l'apprentissage ne débute, la fonction Q est initialisée arbitrairement. Ensuite, à chaque choix d'action, l'agent observe la récompense et le nouvel état (qui dépend de l'état précédent et de l'action actuelle). Le cœur de l'algorithme est une mise à jour de la fonction de valeur selon la formule suivante :

$$Q[s, a] := (1 - \alpha)Q[s, a] + \alpha \left(r + \gamma \max_{a'} Q[s', a'] \right)$$

où s' est le nouvel état, s est l'état précédent, a est l'action choisie, r est la récompense reçue par l'agent, α est un nombre entre 0 et 1 appelé facteur d'apprentissage, et γ est le facteur d'actualisation.

En d'autres termes, la valeur de $Q[s, a]$ est remplacée par une moyenne pondérée entre la précédente valeur de $Q[s, a]$ (pondérée par $1 - \alpha$) et le gain attendu $r + \gamma \max_{a'} Q[s', a']$ (pondérée par α).

Un épisode de l'algorithme finit lorsque s_{t+1} est un état final. Toutefois, le Q-learning peut aussi être appliqué à des tâches non épisodiques. Si le facteur d'actualisation est plus petit que 1, la valeur action-état est finie même pour Δt infini.

N.B. : Pour chaque état final s_f, la valeur de $Q(s_f, a)$ n'est jamais mise à jour et maintient sa valeur initiale. Généralement, $Q(s_f, a)$ est initialisé à zéro.

D Mise en œuvre du Q-learning

1 Architecture du système

Le système est composé de deux modules principaux :

— un planificateur global basé sur RRT qui génère un chemin vers la cible,

— un agent Q-learning qui apprend à suivre et optimiser ce chemin.

Le robot évolue dans un environnement simulé contenant des obstacles.

Le robot évolue dans un environnement simulé en deux dimensions contenant plusieurs obstacles rectangulaires. Une marge de sécurité est ajoutée autour de ces obstacles afin de simuler des collisions réalistes.

Algorithm 4 Q-learning

Entrée : taux d'apprentissage $\alpha > 0$, paramètre d'exploration $\varepsilon > 0$, facteur d'actualisation $\gamma > 0$
Sortie : fonction de valeur $Q[s, a]$
Initialiser $Q[s, a]$ arbitrairement pour tout état s non terminal et toute action a
$Q[s_f, a] \leftarrow 0$ pour tout état terminal s_f et toute action a
for chaque épisode **do**
    Initialiser l'état s
    while s n'est pas terminal **do**
        Choisir une action a depuis s selon une politique ε-greedy basée sur Q
        Exécuter l'action a
        Observer la récompense r et le nouvel état s'
        $Q[s, a] \leftarrow Q[s, a] + \alpha\Big(r + \gamma \max_{a'} Q[s', a'] - Q[s, a]\Big)$
        $s \leftarrow s'$
    end while
end for
return Q

Représentation de l'état L'état s est obtenu par discrétisation de plusieurs variables décrivant la situation du robot :

— la distance au prochain waypoint,
— l'angle entre l'orientation du robot et la direction de la cible,
— la vitesse du robot.

Cette représentation permet de transformer un espace continu en un ensemble fini d'états compatibles avec une Q-table.

Espace d'actions L'agent dispose de quatre actions possibles :

— accélérer,
— freiner,
— tourner à gauche,
— tourner à droite.

Ces actions influencent directement la dynamique du robot simulé.

2 Processus d'apprentissage

L'apprentissage est réalisé sur plusieurs milliers d'épisodes (jusqu'à 10 000). À chaque épisode, le robot est réinitialisé puis apprend à suivre le chemin généré par le planificateur RRT à l'aide du Q-learning.

Les transitions (s, a, r, s') observées au cours des épisodes sont utilisées pour mettre à jour la Q-table selon la règle d'apprentissage du Q-learning.

Une stratégie ε-greedy est utilisée afin d'équilibrer :

— l'exploration de nouvelles actions,
— l'exploitation des connaissances déjà apprises.

Afin d'accélérer l'apprentissage, une phase d'entraînement est réalisée sans affichage graphique. Dans cette phase, aucun rendu visuel n'est effectué et seule la Q-table est mise à jour. Cette approche permet d'exécuter rapidement un grand nombre d'épisodes.

À la fin de l'entraînement, la Q-table est sauvegardée dans un fichier `best_qtable.npy` puis réutilisée lors de la phase de visualisation.

Le système est ainsi organisé en deux phases :

— une phase d'apprentissage hors affichage,

— une phase de visualisation utilisant la politique apprise.

Lors de la visualisation, le paramètre d'exploration ε est réduit afin de privilégier l'exploitation des connaissances apprises.

3 Fonction de récompense

La fonction de récompense est conçue afin de guider progressivement le robot vers un comportement optimal.

Les principales récompenses utilisées sont :

— une forte pénalité en cas de collision (-100),

— une récompense importante lorsque l'objectif est atteint ($+100$),

— une récompense progressive liée à la réduction de distance vers la cible,

— une récompense supplémentaire lors de l'utilisation de raccourcis sur le chemin RRT,

— une pénalité pour les changements brusques d'orientation.

Cette fonction de récompense favorise à la fois :

— l'évitement des obstacles,

— la fluidité des trajectoires,

— la réduction du temps de navigation.

4 Expérimentations

Les résultats de l'apprentissage sont enregistrés dans des fichiers CSV contenant notamment :

— la récompense totale par épisode,

— le nombre de pas nécessaires pour atteindre l'objectif.

L'analyse des trajectoires montre une amélioration progressive du comportement du robot au cours de l'apprentissage :

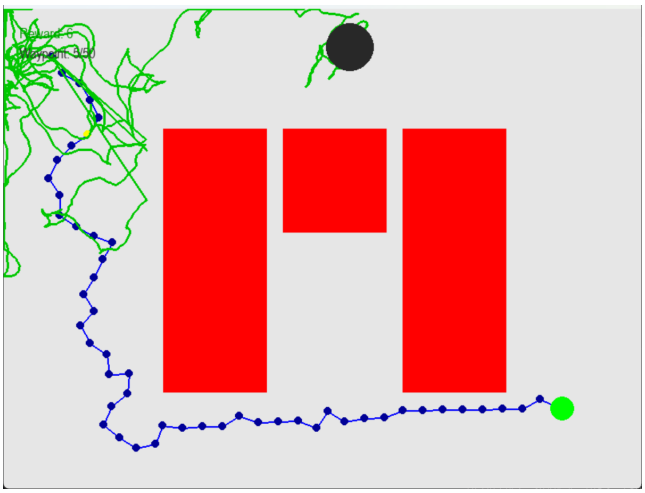


FIGURE 8 – Trajectoire du robot avant apprentissage.

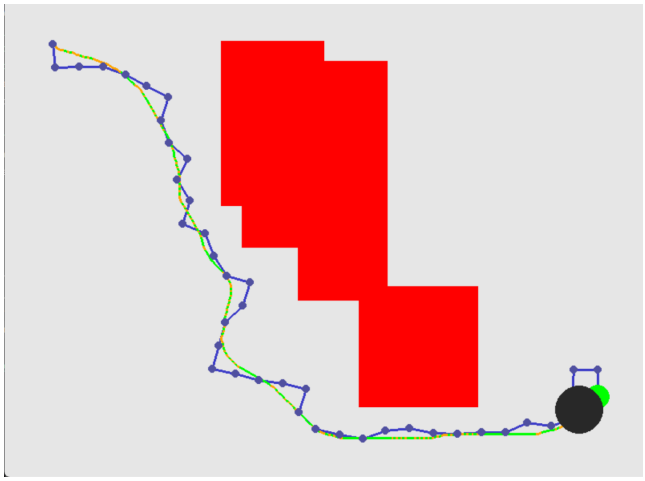


FIGURE 9 – Trajectoire du robot après apprentissage.

Avant entraînement :
— trajectoires instables,
— oscillations importantes,
— collisions fréquentes.
Après apprentissage :
— trajectoires plus fluides,
— meilleure gestion des virages,
— réduction des oscillations,
— navigation plus stable et efficace.
La trajectoire du robot est également visualisée avec un code couleur dépendant de sa vitesse :
— rouge : vitesse faible,
— orange : vitesse moyenne,
— vert : vitesse élevée.
Cette représentation met en évidence :
— le ralentissement à proximité des obstacles,
— l'accélération dans les zones dégagées,
— l'amélioration de la fluidité après apprentissage.

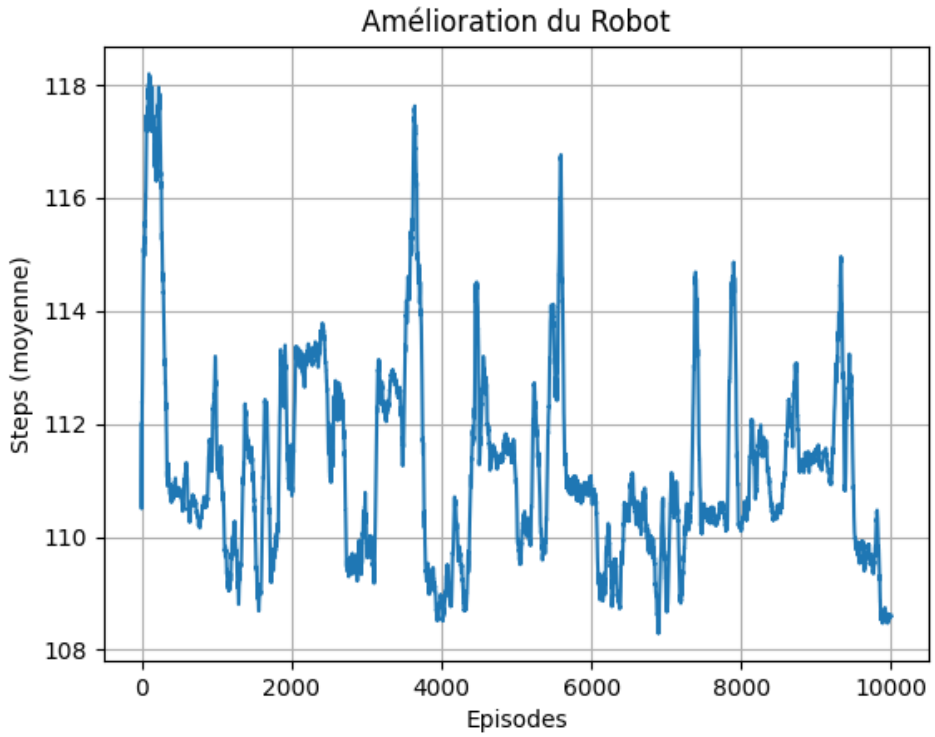


FIGURE 10 – Évolution du nombre moyen de pas nécessaires pour atteindre l'objectif au cours de l'apprentissage.

La figure 10 montre l'évolution du nombre moyen de pas nécessaires pour atteindre la cible en fonction du nombre d'épisodes d'apprentissage.

Au début de l'entraînement, les performances du robot sont instables en raison de la phase d'exploration du Q-learning. Le robot teste différentes actions et génère donc des trajectoires peu optimisées.

Au fil des épisodes, on observe la diminution du nombre de pas moyens, ce qui indique une amélioration progressive de la politique apprise par l'agent.

Les fluctuations visibles sur la courbe sont dues à la stratégie d'exploration ε-greedy ainsi qu'à la complexité de l'environnement contenant des obstacles. Malgré ces variations, la courbe reste globalement décroissante, traduisant une convergence progressive du processus d'apprentissage.

Cette amélioration se manifeste également par des trajectoires plus fluides, plus directes et nécessitant moins de corrections de direction.

E Analyse et discussion

Les résultats obtenus peuvent être interprétés à travers les mécanismes internes du Q-learning et la structure de l'environnement de navigation.

1 Impact de l'exploration et de l'exploitation

Au début de l'entraînement, l'agent adopte une politique fortement exploratoire ($\varepsilon \approx 1$), ce qui conduit à des trajectoires aléatoires et à des performances faibles. Cette phase est nécessaire afin de couvrir un maximum d'états dans la Q-table.

Au fil des épisodes, la décroissance de ε favorise progressivement l'exploitation des actions apprises. Cela se traduit par une amélioration de la stabilité des trajectoires et une réduction du nombre de pas nécessaires pour atteindre l'objectif.

2 Convergence de la fonction de valeur

La Q-table converge progressivement vers des valeurs cohérentes grâce à la mise à jour :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big(r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big)$$

Cette mise à jour permet de renforcer les actions conduisant à des états plus proches de l'objectif. La convergence est toutefois influencée par :
— la taille de la discrétisation de l'espace d'état,
— le choix du facteur d'apprentissage α,
— le facteur de discount γ.

3 Effet de la fonction de récompense

La fonction de récompense joue un rôle central dans la qualité de l'apprentissage. Elle combine plusieurs objectifs :
— **Progression vers la cible :** récompense proportionnelle à la réduction de distance,
— **Pénalisation des collisions :** forte pénalité (-100) assurant l'évitement des obstacles,
— **Fluidité de mouvement :** pénalisation de l'orientation afin d'éviter les trajectoires instables,
— **Raccourcis (shortcuts) :** récompense additionnelle lorsque l'agent détecte un waypoint plus avancé.

Ce dernier point explique la tendance du robot à "lisser" les trajectoires en sautant certains points du chemin RRT lorsque cela est possible.

4 Amélioration des trajectoires

L'évolution des performances observée dans les données CSV (augmentation de la récompense moyenne et diminution du nombre de pas) indique une optimisation progressive de la politique.

Cette amélioration se manifeste visuellement par :
— des trajectoires plus directes,
— une réduction des oscillations autour des obstacles,
— une meilleure exploitation des raccourcis dans le chemin RRT.

5 Limites du modèle

Malgré les améliorations observées au cours de l'apprentissage, plusieurs limites du Q-learning apparaissent dans ce type d'environnement.

Le Q-learning tabulaire devient difficilement applicable lorsque l'espace d'états S est très grand, voire continu. En effet, la taille de la Q-table croît rapidement avec le nombre d'états et d'actions possibles, ce qui entraîne :
— une consommation mémoire importante,
— un temps d'apprentissage élevé,
— une exploration incomplète de l'espace d'états,
— une convergence plus lente de la politique.

Dans notre problème de navigation robotique, les variables physiques telles que la position, l'orientation ou la vitesse sont naturellement continues. Une discrétisation est donc nécessaire afin de rendre le problème compatible avec une Q-table classique. Cependant, cette discrétisation peut entraîner une perte de précision et limiter la qualité de la politique apprise.

De plus, les performances du modèle restent fortement dépendantes du choix des hyperparamètres, notamment :
— le facteur d'apprentissage α,
— le facteur d'exploration ε,
— le facteur d'actualisation γ.

Un mauvais réglage de ces paramètres peut conduire à un apprentissage instable, à des oscillations de la politique ou à une convergence prématurée vers une solution sous-optimale.

Enfin, dans des environnements complexes ou continus, il devient généralement nécessaire d'utiliser des méthodes d'approximation de fonction, comme les réseaux de neurones profonds utilisés dans les Deep Q-Networks (DQN), afin de remplacer la représentation tabulaire classique.

IV Apprentissage par renforcement profond : Deep Q-Learning (DQN)

A Introduction au DQN

Comme nous l'avons vu avec notre encadrant et dans l'ouvrage *Acting, Planning, and Learning*, le Q-Learning classique fonctionne très bien pour de petits environnements. Cependant, dans notre simulateur, le robot évolue dans un espace continu avec des positions précises, un angle et une vitesse. Si on voulait faire une Q-Table pour stocker toutes ces situations possibles, la taille serait infinie et ça ne fonctionnerait pas.

C'est pourquoi nous sommes passés au **Deep Q-Learning (DQN)**. L'idée est d'utiliser un **réseau de neurones** à la place du tableau. Le réseau prend en entrée ce que "voit" le robot et

prédit quelle action est la meilleure à faire dans cette situation. Ce n'est donc plus une table qu'on remplit case par case, mais un réseau qui apprend à partir de ses expériences passées.

Toutefois, lors de nos premiers essais, nous avons fait face à des comportements inattendus. Notre robot avait par exemple tendance à tourner en rond sur lui-même à l'infini (il faisait l'hélicopter). De son point de vue, bouger risquait de le faire heurter un mur et prendre une pénalité, donc rester sur place était la stratégie la plus sûre. Pour corriger ça et rendre l'apprentissage plus stable, deux mécanismes sont indispensables :

— **La mémoire :** Le robot stocke ses actions passées. Au lieu d'apprendre uniquement de ce qu'il vient de faire, il s'entraîne sur des lots de situations tirées aléatoirement dans sa mémoire. Cela lui évite d'oublier comment esquiver un mur juste parce qu'il roule en ligne droite depuis un moment.

— **Le Double DQN :** On utilise deux réseaux de neurones. Un premier réseau *joueur* qui choisit les actions et apprend puis un second réseau *arbitre* mis à jour moins souvent, qui sert de référence pour juger si une action était vraiment bonne. Sans cet arbitre, les deux rôles se mélangent et l'apprentissage devient instable.

La formule de mise à jour reste celle de Bellman, mais appliquée au réseau :

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big(r + \gamma \max_{a'} Q_{\text{arbitre}}(s',a') - Q(s,a)\Big)$$

où r est la récompense reçue, γ le facteur d'actualisation, et α le taux d'apprentissage.

B Contribution (Notre approche et évolution du modèle)

Notre objectif était de faire collaborer le chemin calculé par le RRT (le GPS du robot) avec l'intelligence du DQN (le pilote). Pour y arriver, notre approche a évolué en plusieurs étapes, chacune à caude de problème concret que l'on a observé pendant l'entraînement.

1 L'évolution de la perception du robot

Au départ, nous avons simplement entraîné le robot à suivre les points du chemin RRT. Le problème est qu'il suivait la ligne sans faire attention à son environnement et rentrait dans les murs dès la moindre erreur de trajectoire, n'ayant aucune notion des obstacles autour de lui.

Pour lui apprendre à les éviter, nous avons d'abord ajouté 3 capteurs LIDAR orientés vers l'avant (-30, 0, 30). C'est un peu comme des yeux qui mesurent la distance aux obstacles dans ces trois directions.

L'amélioration était bonne mais le robot restait aveugle sur les côtés. En longeant un mur de trop près, il ne le voyait pas et finissait par le heurter. Nous sommes donc finalement passés à **5 capteurs** (-60, -30, 0, 30, 60), ce qui lui offre une meilleure perception.

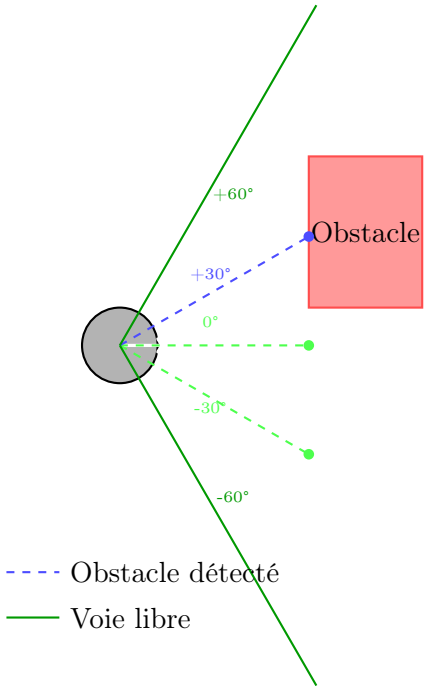


FIGURE 11 – Le robot et ses 5 capteurs LIDAR. Le rayon bleu touche un obstacle et retournent une distance réduite.

2 L'architecture du réseau de neurones

Pour traiter toutes ces informations, nous avons conçu un réseau de neurones avec PyTorch. L'architecture finale est assez simple :

— **Couche d'entrée (13 neurones) :** Elle reçoit 8 informations sur l'état du robot (position, angle, vitesse, distance et direction vers le prochain point RRT) ainsi que les 5 valeurs des capteurs LIDAR. Toutes ces valeurs sont normalisées entre 0 et 1 pour que le réseau puisse les traiter correctement.

— **Deux couches intermédiaires (128 neurones chacune) :** Ce sont les couches qui font le vrai travail d'apprentissage. Elles utilisent une fonction d'activation ReLU. Nous avons choisi 128 en taille car c'est assez grand pour apprendre des comportements complexes et assez petite pour ne pas prendre des heures à entraîner.

— **Couche de sortie (5 neurones) :** Elle donne un score pour chacune des 5 actions possibles : accélérer, freiner, tourner à gauche, tourner à droite, ou ne rien faire. Le robot choisit toujours l'action avec le score le plus élevé.

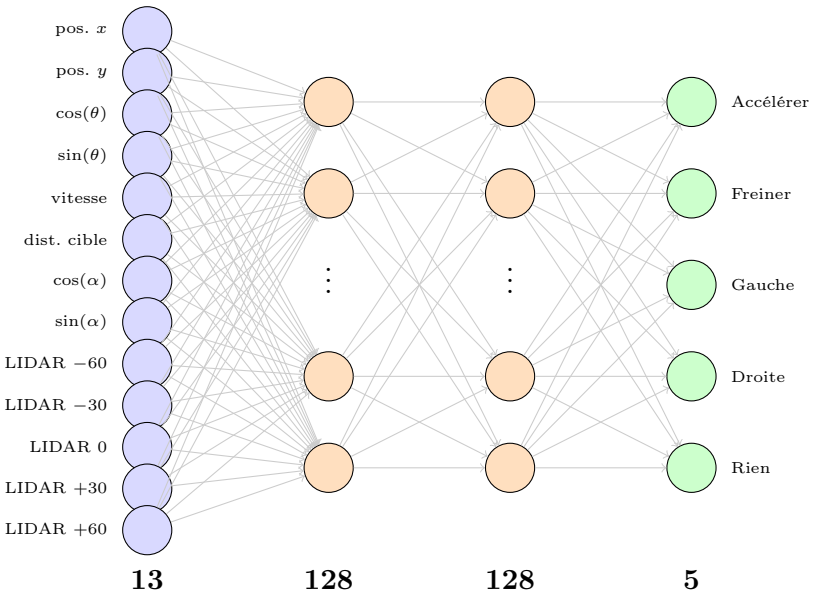


FIGURE 12 – Architecture du réseau DQN utilisé. La couche d'entrée reçoit les 8 variables d'état du robot et les 5 mesures LIDAR. Les deux couches intermédiaires utilisent une activation ReLU. La couche de sortie produit un score pour chacune des 5 actions possibles.

3 Le gonflement des obstacles

Nous avons rencontré un problème assez concret : le RRT calcule un chemin pour un point sans taille, mais notre robot fait 25 pixels de rayon. Du coup, le RRT traçait parfois des chemins collés aux murs, physiquement impossibles à suivre pour le robot.

Pour régler ça, on a trouvé une astuce simple : avant de lancer le RRT, on gonfle virtuellement tous les obstacles d'une marge correspondant au rayon du robot plus une zone de sécurité. Le RRT est ainsi forcé de planifier un chemin qui passe bien au milieu des couloirs, laissant de la place au robot pour avancer.

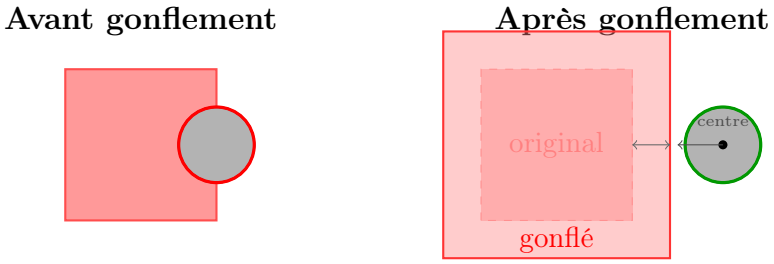


FIGURE 13 – Principe du gonflement des obstacles. À gauche : le robot dont le bord touche l'obstacle, ce qui génère une collision. À droite : l'obstacle est élargi du rayon du robot ; il suffit alors de tester si le **centre** du robot touche l'obstacle gonflé.

4 Le système de récompenses

Définir les bonnes récompenses a été l'une des parties les plus longues du projet. Si les récompenses ne sont pas bien calibrées, le robot trouve des "raccourcis" pour avoir des points sans vraiment faire ce qu'on veut. Le système final est :

— **Guidage :** +100 à chaque waypoint RRT atteint, +200 à l'arrivée finale.
— **Pénalité de temps :** −0.1 à chaque pas pour éviter qu'il reste immobile.
— **Récompense d'approche :** petit bonus si le robot se rapproche de sa cible, petit malus s'il s'en éloigne.
— **Récompense d'alignement :** petit bonus si le robot regarde dans la bonne direction.
— **Pénalité murs :** si un capteur voit un obstacle à moins de 40 % de sa portée, le robot prend une pénalité plus forte que s'il est proche. Et si en plus il va vite, la pénalité est encore plus grande pour l'obliger à freiner à l'avance.
— **Collision :** −200 et fin de l'épisode immédiatement.

5 La mécanique de traumatisme

On a réalisé que dans la mémoire du robot, les collisions étaient rares par rapport aux milliers de transitions normales. Du coup, il ne les revoyait pas assez souvent pour vraiment les retenir. Pour corriger ça, chaque fois qu'une collision se produit, on enregistre 10 fois cette transition dans la mémoire. Ça force le réseau à beaucoup mieux mémoriser ce genre d'événement et à l'éviter plus efficacement.

C Expérimentation (Analyse et résultats)

1 Protocole d'entraînement

Pour éviter que le robot apprenne un trajet par cœur, on le fait apparaître à des positions aléatoires sur 5 cartes différentes à chaque épisode, avec un chemin RRT recalculé à chaque fois. Le tableau ci-dessous résume les principaux paramètres utilisés pour l'entraînement final.

Paramètre	**Valeur**
Taille de la mémoire	20 000 transitions
Taille des lots d'entraînement	128
Taux d'apprentissage	0.001
Facteur d'actualisation	0.99
Exploration initiale	100 % aléatoire
Exploration finale	5 % aléatoire
Mise à jour du réseau arbitre	tous les 50 épisodes
Nombre d'épisodes	3 000
Limite de pas par épisode	800

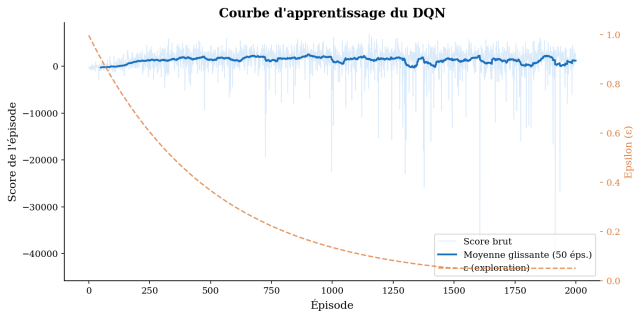
Table 1 – Paramètres d'entraînement du DQN

2 Progression de l'entraînement

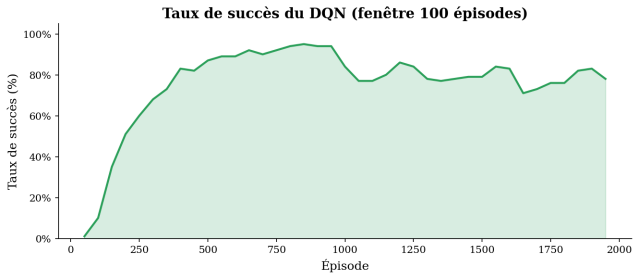
Nous avons eu plusieurs versions d'entraînement avant d'arriver au résultat final. On les présente ici pour montrer comment le modèle a évolué au fil des corrections.

Version 1 — Récompenses mal calibrées Dans cette première version (2 000 épisodes), le problème venait du fait que les pénalités liées aux capteurs étaient beaucoup trop faibles par rapport aux récompenses d'arrivée. Concrètement, le robot recevait +100 quand il atteignait un waypoint, mais seulement quelques points négatifs quand il frôlait un mur. Du point de vue du

robot, foncer vers l'objectif en ignorant les murs était donc toujours plus rentable que d'être prudent. Il n'avait aucune raison d'apprendre à les éviter correctement.



(a) Score au fil des épisodes — Version 1

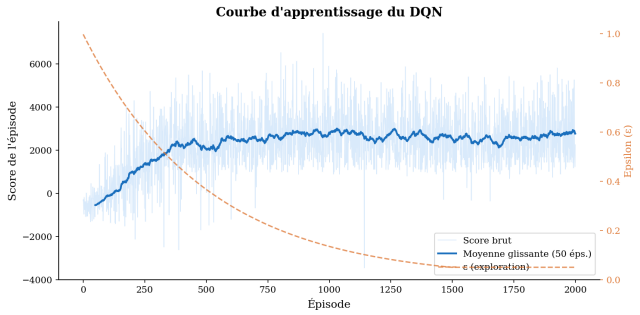


(b) Taux de réussite — Version 1

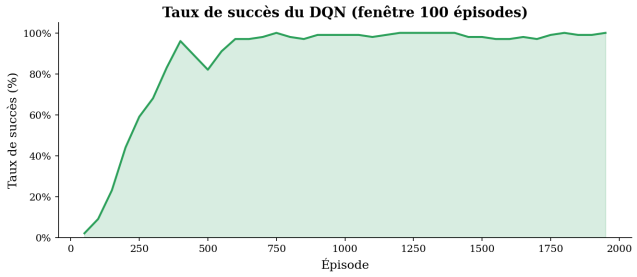
FIGURE 14 – Résultats de la première version (2 000 épisodes). Les scores restent proches de zéro sans vraiment progresser, et le taux de réussite plafonne entre 80 et 95 % sans se stabiliser, ce qui indique que l'apprentissage est bloqué.

Comme on peut le voir sur la figure 14, les scores n'augmentent pas vraiment malgré un taux de réussite correct. En fait le robot arrivait à destination mais prenait des pénalités à chaque fois qu'il longeait un mur, ce qui explique la moyenne qui stagne.

Version 2 — Récompenses corrigées Pour la deuxième version, on a corrigé le problème de limitation des récompenses en augmentant la pénalité de collision à -200 (qui est directement appliquée sans limitation), on a renforcé les pénalités liées aux capteurs, et on a aussi corrigé la façon dont les points de départ et d'arrivée sont générés pour éviter des situations où le RRT ne trouve pas de chemin.



(a) Score au fil des épisodes — Version 2

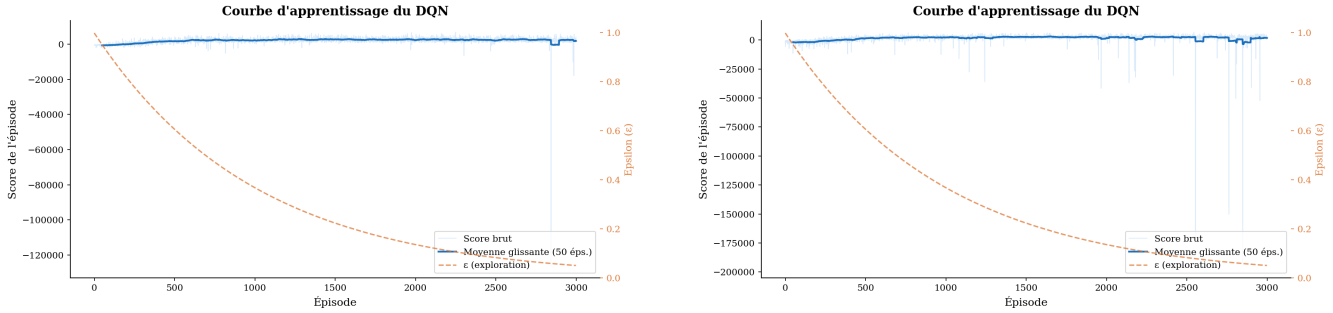


(b) Taux de réussite — Version 2

FIGURE 15 – Résultats de la deuxième version (2 000 épisodes, récompenses corrigées). Le score monte progressivement jusqu'à environ 2 800 et le taux de réussite atteint 100 % vers l'épisode 600.

La figure 15 montre une amélioration très claire. Le score progresse de façon continue, ce qui confirme que le robot apprend bien. Le taux de réussite à 100 % montre qu'il arrive à atteindre l'arrivée sur toutes les cartes d'entraînement.

Version 3 — Entraînement sur 3 000 épisodes avec traumatisme Pour la version finale, on a allongé l'entraînement à 3 000 épisodes et ajouté la mécanique de traumatisme. On a cependant observé que certaines sessions produisaient des scores très négatifs sur quelques épisodes (figure 16), à cause de l'interaction entre les fortes pénalités des capteurs et la duplication des collisions dans la mémoire.

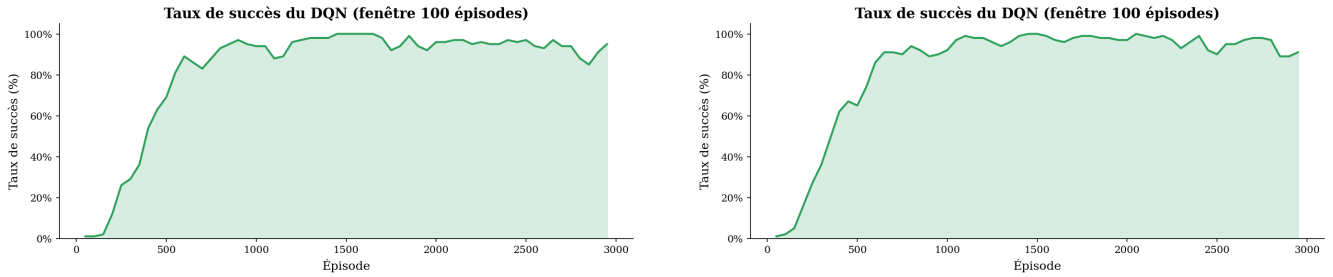


(a) Session avec forte variance

(b) Session qui converge normalement

FIGURE 16 – Deux sessions d'entraînement de la version 3 (3 000 épisodes). La session de gauche montre des pics très négatifs sur certains épisodes, là où le robot longe un mur pendant longtemps et accumule les pénalités capteurs. La session de droite avec les mêmes paramètres se passe mieux, ce qui montre que l'aléatoire des positions de départ joue un rôle important.

Malgré ces épisodes difficiles, le taux de réussite global reste très bon sur les deux sessions comme on peut le voir sur la figure 17.



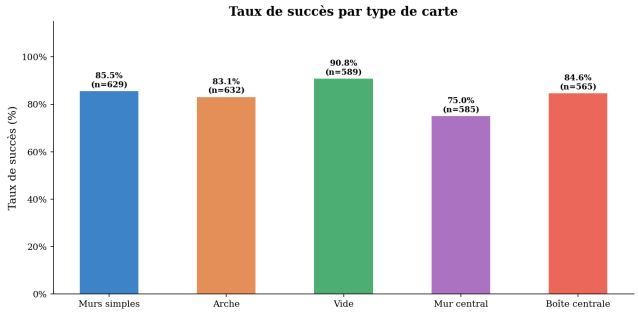
(a) Taux de réussite — session A

(b) Taux de réussite — meilleure session

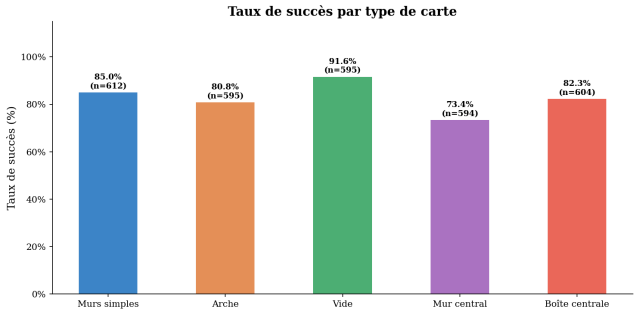
FIGURE 17 – Taux de réussite de la version 3 sur 3 000 épisodes. Le robot dépasse 90 % dès l'épisode 500 environ et se maintient entre 90 et 100 % sur la durée.

3 Résultats par carte

La figure 18 montre le taux de réussite séparé par carte, ce qui nous permet de voir si le robot se débrouille bien sur toutes les configurations ou s'il a appris à en maîtriser que quelques-une.



(a) Version 2 (2 000 épisodes)

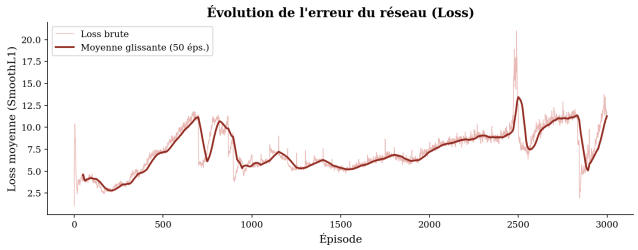


(b) Version 3 (3 000 épisodes)

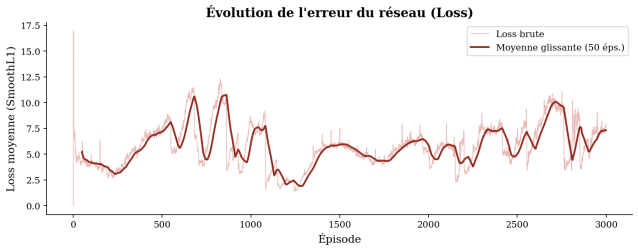
FIGURE 18 – Taux de réussite par type de carte. Les valeurs n indiquent le nombre d'épisodes joués sur chaque carte.

Sans surprise, la carte **Vide** (aucun obstacle) est la plus facile avec environ 91 % de réussite. La carte **Mur central** est la plus difficile avec seulement 73–75 % : le robot doit longer le mur sur une longue distance pour le contourner, ce qui est une situation compliquée pour les capteurs. Dans l'ensemble, les taux restent corrects sur toutes les cartes, ce qui montre que le robot a bien appris et pas juste à mémoriser quelques situations.

4 Évolution de l'erreur du réseau



(a) Erreur du réseau — Version 2



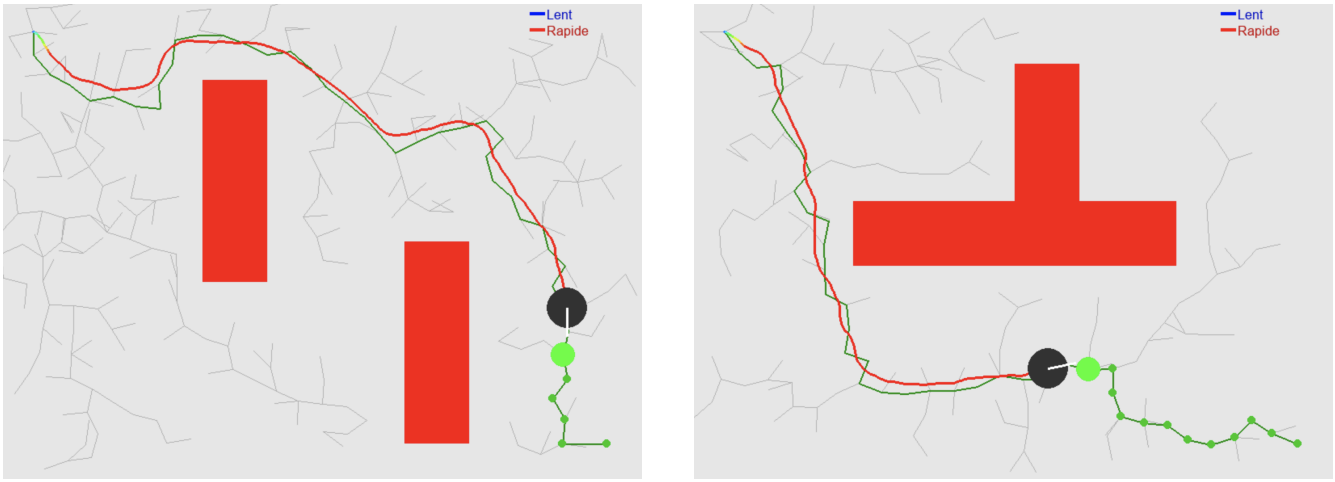
(b) Erreur du réseau — Version 3

FIGURE 19 – Évolution de l'erreur du réseau au fil des épisodes.

L'erreur du réseau (figure 19) n'a pas tendance à descendre vers zéro comme on pourrait s'y attendre. On pense que c'est normal car le réseau essaie de prédire des valeurs qui changent elles-mêmes au fil de l'apprentissage. Les oscillations reflètent les moments où de nouveaux comportements sont explorés puis pour les baisses, les moments où le réseau applique ce qu'il a appris.

5 Visualisation du comportement

Pour visualiser concrètement ce que le robot a appris, nous avons affiché sa trajectoire réelle en jeu, superposée au chemin planifié par le RRT.



(a) Chemin réussi (b) Contournement d'obstacle

FIGURE 20 – Comportement du robot après entraînement, sur une carte non vue pendant l'apprentissage. En vert le chemin planifié par le RRT, en gris l'arbre de recherche, en bleu–rouge la trajectoire réellement suivie selon la vitesse.

On peut voir sur la figure 20 que le robot suit globalement le chemin prévu par le RRT tout en gérant ses propres déplacements. Ce qui est intéressant c'est qu'il a appris à rouler à pleine vitesse la plupart du temps, il a bien compris que trainer en route lui coûtait des points. Ça confirme que la pénalité de temps a bien joué son rôle.



FIGURE 21 – Exemple d'échec sur la carte Boîte centrale. Le robot tente de longer le couloir étroit mais heurte le mur. Ce type de situation explique le taux de réussite plus faible observé sur cette carte (73–75 %).

D Pistes d'amélioration

Les résultats obtenus sont encourageants car le robot atteint des taux de réussite de 73 à 91 % selon la difficulté de la carte. Mais il y a des choses qui pourraient être améliorées si on avait plus

de temps et de ressources.

La première piste serait d'**augmenter le nombre d'épisodes d'entraînement**. Avec 3 000 épisodes on voit déjà une bonne progression, mais tourner 10 000 épisodes ou plus permettrait peut-être au robot de mieux gérer les cartes difficiles comme le mur central. Le problème c'est que sur nos machines personnelles ça prendrait beaucoup trop de temps.

Ensuite, notre encadrant nous a suggéré d'utiliser aléatoirement les différentes variantes de RRT (RRT classique, RRTSC, RRT*) lors de l'entraînement. Notre robot s'entraîne uniquement sur des chemins calculés par le RRT de base. Si on lui présentait aussi des chemins issus du RRTSC ou du RRT* (qui sont plus optimaux et plus lisses), il devrait apprendre à suivre des chemins de qualité variable, ce qui le rendrait bien plus robuste (nous lui avons proposés un waypoint de RRT* mais il n'arrive pas à suivre et à adapter sa vitesse pour des séquences de point trop éloigné).

Une autre idée serait de générer les cartes de manière aléatoire pendant l'entraînement au lieu de se limiter à nos 5 cartes. Un nombre aléatoire d'obstacles, à des positions et avec des tailles aléatoires. Cela forcerait le robot à vraiment tout comprendre et pas juste à s'adapter à une petite sélection de configurations qu'on lui a donné.

On pourrait aussi tester différents taux d'apprentissage pour voir lequel donne les meilleurs résultats. Un taux trop élevé peut rendre l'apprentissage instable et un taux trop faible peut le rendre très lent. Comparer plusieurs valeurs (par exemple 0.0001, 0.001 et 0.01) permettrait de mieux calibrer ces paramètre.

Notre simulateur aussi, est déterministe. Quand le robot décide d'accélérer, il accélère toujours exactement comme prévu. Dans la réalité, un robot sur un sol glissant ou avec des capteurs imprécis ne se comporterait pas toujours comme prévu donc on pourrait introduire un peu d'aléatoire dans les actions (par exemple de légère dérive aléatoire) serait un bon moyen d'entraîner le robot à être plus robuste face à l'incertain mais ça nous demanderait beaucoup plus de ressource de calcul et nous n'avons pas de supercalculateur.

Conclusion

Ce projet nous a permis de répondre à notre question de départ : il est possible d'exploiter une trajectoire issue d'un algorithme de planification déterministe pour guider efficacement un agent d'apprentissage par renforcement dans un environnement incertain. La combinaison **RRT + DQN** et le planificateur donnent une route structurée que le contrôleur apprend à suivre de manière autonome et robuste.

D'abord, le **Q-learning** tabulaire atteint rapidement ses limites face à un espace d'états continu, ce qui rend le passage au **DQN** indispensable. Ensuite, le calibrage de la fonction de récompense s'est avéré être l'une des étapes les plus délicates du projet : une mauvaise calibration conduit le robot à exploiter des comportements non désirés plutôt qu'à apprendre ce qu'on attend de lui. Enfin, notre simulateur reste déterministe et simplifié, ce qui constitue un écart notable avec les conditions d'un environnement réel.

Les résultats obtenus, avec des taux de réussite entre 73 % et 91 % selon la difficulté des cartes, sont encourageants mais laissent une marge de progression. Plusieurs pistes mériteraient d'être explorées : augmenter le nombre d'épisodes d'entraînement, diversifier les planificateurs utilisés durant l'apprentissage, générer des cartes aléatoires pour améliorer la généralisation, et introduire du bruit dans la simulation pour se rapprocher des conditions réelles.

Références

[1] S. M. LaValle, *Rapidly-exploring Random Trees : A New Tool for Path Planning*, 1998.

[2] James J. Kuffner Jr. *Randomized Kinodynamic Planning*, 2001

[3] Sertac Karaman, Emilio Frazzoli. *Incremental sampling-based algorithms for optimal motion planning*, 2010.

[4] Wikipédia, Cover tree
`https://en.wikipedia.org/wiki/Cover_tree`

[5] M. Ghallab, D. Nau, P. Traverso, *Acting, Planning, and Learning*, LAAS-CNRS, 2016. `https://projects.laas.fr/planning/apl.pdf`

[6] PyTorch, *Reinforcement Learning (DQN) Tutorial*.
`https://docs.pytorch.org/tutorials/intermediate/reinforcement/q/learning.html`

[7] Wikipédia, *Q-learning*.
`https://fr.wikipedia.org/wiki/Q-learning`

Le résumé

Résumé (français)

Ce projet étudie la navigation autonome d'un robot mobile dans un environnement 2D incertain en combinant planification de trajectoire et apprentissage par renforcement afin d'obtenir un contrôleur robuste. Trois variantes de **RRT** sont comparées : **RRT**, **RRT*** et **RRTSC**, cette dernière étant une amélioration originale intégrant une phase de raccourcis. Le **RRTSC** offre le meilleur compromis entre rapidité et qualité des trajectoires.

Dans un premier temps, un agent de **Q-learning** est utilisé mais montre ses limites dans un espace d'états continu, ce qui motive l'adoption du **DQN**, basé sur des réseaux de neurones. Le robot est équipé de capteurs **LIDAR** et l'apprentissage est stabilisé grâce à des techniques telles que la mémoire de rejeu, le **Double DQN** et une mécanique de traumatisme.

Les résultats montrent des taux de réussite compris entre 73 % et 91 % selon la complexité des cartes, validant l'intérêt de combiner **planification** et **apprentissage**.

Summary (English)

This project investigates autonomous navigation of a mobile robot in an uncertain 2D environment by combining path planning and reinforcement learning to achieve a robust controller. Three variants of **RRT** are compared : **RRT**, **RRT***, and **RRTSC**, the latter being an original extension that introduces a shortcut post-processing phase. **RRTSC** provides the best trade-off between speed and path quality.

Initially, a **Q-learning** agent is used but struggles with continuous state spaces, leading to the adoption of **DQN**, which relies on neural networks. The robot is equipped with **LIDAR** sensors, and training stability is improved using techniques such as experience replay, **Double DQN**, and a trauma mechanism.

Results show success rates between 73 % and 91 % depending on map complexity, demonstrating the effectiveness of combining **planning** and **learning**.